IBEX

Trekking Platform

Standard Operating Procedure

Complete step-by-step guide to design, build, and deploy

Stack: Node.js · MongoDB · Next.js · Razorpay · Cloudinary · Nodemailer · Google OAuth 2.0

Version 1.0 · June 2026

Next.js	Node.js / Express	MongoDB	Razorpay	Cloudinary	Nodemailer	Google OAuth

TABLE OF CONTENTS

PHASE 1 · PROJECT SETUP

Environment Configuration

IBEX Trekking — Standard Operating Procedure

1.1 Folder Structure

Follow this mono-repo structure — backend and frontend in one repository, clearly separated.

```
ibex/
■■■ backend/
■ ■■■ config/ # db.js, cloudinary.js, passport.js
■ ■■■ controllers/ # authController, trekController, bookingController …
■ ■■■ middleware/ # authMiddleware.js, uploadMiddleware.js
■ ■■■ models/ # User.js, Trek.js, Booking.js, Payment.js, Blog.js, Review.js
■ ■■■ routes/ # auth.js, treks.js, bookings.js, payments.js, blogs.js
■ ■■■ services/ # emailService.js, razorpayService.js, cloudinaryService.js
■ ■■■ utils/ # generateToken.js, signatureVerify.js, sendEmail.js
■ ■■■ .env
■ ■■■ server.js
■■■ frontend/ # Next.js app (App Router)
■■■ app/
■■■ components/
■■■ lib/ # api.js (Axios instance)
■■■ public/
■■■ .env.local
```

1.2 Backend Initialization

1	**Create backend folder and init npm** mkdir ibex && cd ibex && mkdir backend && cd backend && npm init -y
2	**Install core dependencies** npm i express mongoose dotenv cors cookie-parser helmet morgan passport passport-google-oauth20 jsonwebtoken bcryptjs multer cloudinary nodemailer razorpay express-rate-limit express-validator
3	**Install dev dependencies** npm i -D nodemon jest supertest

4	**Create server.js** Entry point: import express, connect MongoDB, register all route files, attach error handler middleware, listen on PORT from .env.
5	**Create .env file** Add all environment variables — see Appendix (Phase 9) for the complete list.

1.3 Frontend Initialization

1	**Scaffold Next.js app** cd .. && npx create-next-app@latest frontend --app --tailwind --eslint
2	**Install frontend packages** npm i axios @tanstack/react-query react-hook-form razorpay next-auth
3	**Configure next.config.js** Add image domains: ['res.cloudinary.com', 'lh3.googleusercontent.com'] for remote image optimization.
4	**Create lib/api.js** Axios instance with baseURL=process.env.NEXT_PUBLIC_API_URL and withCredentials: true so the httpOnly cookie is sent on every request.

PHASE 2 · DATABASE DESIGN

MongoDB + Mongoose Models

IBEX Trekking — Standard Operating Procedure

2.1 MongoDB Atlas Setup

1	**Create Atlas cluster** Go to cloud.mongodb.com → New Project 'IBEX' → Free M0 cluster → region: Mumbai (ap-south-1).
2	**Create database user** Security → Database Access → Add user with password auth. Role: readWriteAnyDatabase.
3	**Whitelist IP** Network Access → Add IP → 0.0.0.0/0 for development. Restrict to server IP in production.
4	**Get connection string** Clusters → Connect → Drivers → Copy URI. Paste in .env as MONGO_URI.

2.2 Mongoose Models

Create one file per model in backend/models/. Key schema decisions are shown below.

User.js

- name: String (required)
- email: String (unique, required)
- googleId: String (unique, sparse)
- avatar: String (Cloudinary URL)
- role: String (enum: ['user','admin'], default: 'user')
- createdAt: Date (default: Date.now)

Trek.js

- title, slug (unique), region, difficulty
- duration_days: Number, altitude_ft: Number
- price_inr: Number, maxGroupSize: Number
- itinerary: [{day: Number, title, description}]
- images: [String] — Cloudinary URLs
- highlights: [String], included: [String], excluded: [String]
- active: Boolean (default: true)

Booking.js

• userId: ObjectId (ref: 'User', required)

• trekId: ObjectId (ref: 'Trek', required)

• trekDate: Date, groupSize: Number

• totalAmount: Number

• status: enum ['pending','confirmed','cancelled']

• paymentId: ObjectId (ref: 'Payment')

Payment.js

• bookingId: ObjectId (ref: 'Booking')

• rzp_order_id: String (required)

• rzp_payment_id: String

• rzp_signature: String

• amount: Number, currency: String (default: 'INR')

• status: enum ['created','captured','failed']

• paidAt: Date

Blog.js

• title, slug (unique), content (rich HTML)

• tags: [String], category: String

• coverImage: String (Cloudinary URL)

• author: String, publishedAt: Date

• active: Boolean

Review.js

• userId: ObjectId (ref: 'User')

• trekId: ObjectId (ref: 'Trek')

• rating: Number (min:1, max:5)

• comment: String

• createdAt: Date

2.3 DB Connection (config/db.js)

```
const mongoose = require('mongoose');

const connectDB = async () => {
try {
await mongoose.connect(process.env.MONGO_URI);
console.log('MongoDB connected');
```

```
} catch (err) {
console.error(err.message);
process.exit(1);
}
};
module.exports = connectDB;
```

PHASE 3 · AUTH SERVICE

Google OAuth 2.0 + JWT + Passport.js

IBEX Trekking — Standard Operating Procedure

3.1 Google Cloud Console Setup

1	**Create a Google Cloud project** console.cloud.google.com → New Project → Name: 'IBEX'
2	**Enable Google+ API** APIs & Services → Library → Search 'Google+ API' → Enable. Also enable 'People API'.
3	**Create OAuth credentials** APIs & Services → Credentials → Create Credentials → OAuth 2.0 Client IDs → Web Application
4	**Set redirect URIs** Authorized redirect URIs: http://localhost:5000/api/auth/google/callback (dev) and https://api.ibex.in/api/auth/google/callback (prod)
5	**Copy credentials** Copy Client ID and Client Secret → paste in backend .env as GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET

3.2 Passport.js Configuration (config/passport.js)

```
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;
const User = require('../models/User');

passport.use(new GoogleStrategy({
clientID: process.env.GOOGLE_CLIENT_ID,
clientSecret: process.env.GOOGLE_CLIENT_SECRET,
callbackURL: process.env.GOOGLE_CALLBACK_URL,
}, async (accessToken, refreshToken, profile, done) => {
try {
let user = await User.findOne({ googleId: profile.id });
if (!user) {
user = await User.create({
googleId: profile.id,
name: profile.displayName,
```

```
email: profile.emails[0].value,
avatar: profile.photos[0].value,
});
}
return done(null, user);
} catch (err) { return done(err, null); }
}));
```

3.3 Auth Routes (routes/auth.js)

```
// GET /api/auth/google
router.get('/google',
passport.authenticate('google', { scope: ['profile', 'email'] }));

// GET /api/auth/google/callback
router.get('/google/callback',
passport.authenticate('google', { session: false, failureRedirect: '/login' }),
(req, res) => {
const token = jwt.sign(
{ userId: req.user._id, role: req.user.role },
process.env.JWT_SECRET,
{ expiresIn: '7d' }
);
res.cookie('jwt', token, {
httpOnly: true,
secure: process.env.NODE_ENV === 'production',
sameSite: 'lax',
maxAge: 7 * 24 * 60 * 60 * 1000 // 7 days
});
res.redirect(process.env.CLIENT_URL + '/dashboard');
}
);

// GET /api/auth/me (protected)
router.get('/me', protect, (req, res) => res.json(req.user));

// POST /api/auth/logout
router.post('/logout', (req, res) => {
res.clearCookie('jwt');
res.json({ message: 'Logged out' });
```

```
});
```

3.4 JWT Auth Middleware (middleware/authMiddleware.js)

```
const protect = async (req, res, next) => {
const token = req.cookies.jwt;
if (!token) return res.status(401).json({ message: 'Not authorized' });
try {
const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = await User.findById(decoded.userId).select('-__v');
next();
} catch { res.status(401).json({ message: 'Token invalid' }); }
};

const adminOnly = (req, res, next) => {
if (req.user?.role !== 'admin')
return res.status(403).json({ message: 'Admin only' });
next();
};
```

■ **Never store the JWT in localStorage — XSS attacks can steal it. Always use httpOnly cookies.**

PHASE 4 · CORE API SERVICES

Trek, Blog & Review Endpoints

IBEX Trekking — Standard Operating Procedure

4.1 Trek Routes (routes/treks.js)

Method	Endpoint	Access	Notes
GET	/api/treks	Public	List all active treks. Supports ?region=&difficulty=&maxDays=
GET	/api/treks/:slug	Public	Single trek detail by slug
POST	/api/treks	Admin only	Create new trek (with Cloudinary image URLs)
PUT	/api/treks/:id	Admin only	Update trek details
DELETE	/api/treks/:id	Admin only	Soft-delete: set active=false
GET	/api/treks/:id/reviews	Public	Get all reviews for a trek
POST	/api/treks/:id/reviews	Protected	Submit a review (1 per user per trek)

4.2 Blog Routes (routes/blogs.js)

Method	Endpoint	Access	Notes
GET	/api/blogs	Public	List blogs. ?category=&tag=&page=&limit=
GET	/api/blogs/:slug	Public	Single blog post by slug
POST	/api/blogs	Admin only	Create blog post
PUT	/api/blogs/:id	Admin only	Update blog
DELETE	/api/blogs/:id	Admin only	Delete blog

4.3 Controller Pattern

Every controller follows the same async/await pattern with centralized error handling.

```
// controllers/trekController.js
exports.getAllTreks = async (req, res) => {
try {
const { region, difficulty, maxDays } = req.query;
const filter = { active: true };
if (region) filter.region = region;
if (difficulty) filter.difficulty = difficulty;
if (maxDays) filter.duration_days = { $lte: Number(maxDays) };
```

```
const treks = await Trek.find(filter).sort('-createdAt');
res.json(treks);
} catch (err) {
res.status(500).json({ message: err.message });
}
};
```

PHASE 5 · MEDIA SERVICE

Cloudinary Image Upload

IBEX Trekking — Standard Operating Procedure

5.1 Cloudinary Account Setup

1	**Create account** cloudinary.com → Sign up (free tier: 25 GB storage, 25 GB bandwidth/month).
2	**Get credentials** Dashboard → API Keys → Copy Cloud Name, API Key, API Secret → paste in .env
3	**Create upload presets** Settings → Upload → Add Upload Preset → Name: 'ibex_treks'. Set folder: 'ibex/treks', transformations: auto format + auto quality.
4	**Create folders** Media Library → Create folders: ibex/treks, ibex/blogs, ibex/avatars

5.2 Cloudinary Config (config/cloudinary.js)

```
const cloudinary = require('cloudinary').v2;
const { CloudinaryStorage } = require('multer-storage-cloudinary');
const multer = require('multer');

cloudinary.config({
cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
api_key: process.env.CLOUDINARY_API_KEY,
api_secret: process.env.CLOUDINARY_API_SECRET,
});

const storage = new CloudinaryStorage({
cloudinary,
params: async (req, file) => ({
folder: 'ibex/treks',
allowed_formats: ['jpg', 'jpeg', 'png', 'webp'],
transformation: [{ width: 1200, crop: 'limit' }, { quality: 'auto' }],
}),
});

exports.upload = multer({ storage });
```

```
exports.cloudinary = cloudinary;
```

5.3 Upload Middleware Usage

```
const { upload } = require('../config/cloudinary');

// In route file — single image
router.post('/', protect, adminOnly, upload.single('coverImage'), createBlog);

// Multiple images (trek gallery, max 6)
router.post('/', protect, adminOnly, upload.array('images', 6), createTrek);

// In controller — Cloudinary URL is in req.file.path
const imageUrl = req.file.path; // e.g. https://res.cloudinary.com/ibex/...
const imageUrls = req.files.map(f => f.path); // for multiple
```

5.4 Delete Image from Cloudinary

```
const { cloudinary } = require('../config/cloudinary');

// Extract public_id from URL and delete
const deleteImage = async (imageUrl) => {
const publicId = imageUrl.split('/').slice(-2).join('/').split('.')[0];
await cloudinary.uploader.destroy(publicId);
};
```

■ Always delete the Cloudinary image when a trek or blog is deleted. Free tier storage is 25 GB — don't let orphaned images accumulate.

PHASE 6 · BOOKING & PAYMENT

Razorpay Integration

IBEX Trekking — Standard Operating Procedure

6.1 Razorpay Account Setup

1	**Create account** dashboard.razorpay.com → Sign up. Complete KYC for live mode (PAN + bank account).
2	**Get API keys** Settings → API Keys → Generate Test Key. Copy Key ID and Key Secret → paste in .env.
3	**Configure webhook** Settings → Webhooks → Add new → URL: https://api.ibex.in/api/webhooks/razorpay → Events: payment.captured, payment.failed → Copy webhook secret.
4	**Test mode** Use test mode keys during development. Test cards are provided in Razorpay docs.

6.2 Booking Routes (routes/bookings.js)

Method	Endpoint	Access	Notes
POST	/api/bookings	Protected	Create booking + Razorpay order. Returns rzp_order_id.
GET	/api/bookings/my	Protected	User's own bookings
GET	/api/bookings/:id	Protected	Single booking detail
PATCH	/api/bookings/:id/cancel	Protected	Cancel booking if >48h before trek date
GET	/api/bookings	Admin only	All bookings with filters
POST	/api/payments/verify	Protected	Verify Razorpay signature, confirm booking
POST	/api/webhooks/razorpay	Public	Razorpay server-to-server webhook (backup)

6.3 Create Booking + Razorpay Order

```
const Razorpay = require('razorpay');
const razorpay = new Razorpay({
key_id: process.env.RAZORPAY_KEY_ID,
key_secret: process.env.RAZORPAY_KEY_SECRET,
});
```

```
exports.createBooking = async (req, res) => {
const { trekId, trekDate, groupSize } = req.body;
const trek = await Trek.findById(trekId);
const totalAmount = trek.price_inr * groupSize;

// 1. Create Razorpay order (amount in paise)
const rzpOrder = await razorpay.orders.create({
amount: totalAmount * 100,
currency: 'INR',
receipt: `rcpt_${Date.now()}`,
});

// 2. Save pending booking in MongoDB
const booking = await Booking.create({
userId: req.user._id, trekId, trekDate, groupSize,
totalAmount, status: 'pending',
});

// 3. Save payment record with rzp_order_id
const payment = await Payment.create({
bookingId: booking._id,
rzp_order_id: rzpOrder.id,
amount: totalAmount, status: 'created',
});
booking.paymentId = payment._id;
await booking.save();

res.json({ rzp_order_id: rzpOrder.id, amount: totalAmount, bookingId: booking._id });
};
```

6.4 Verify Payment + HMAC Signature

```
const crypto = require('crypto');

exports.verifyPayment = async (req, res) => {
const { rzp_order_id, rzp_payment_id, rzp_signature, bookingId } = req.body;

// HMAC-SHA256 verification
const body = rzp_order_id + '|' + rzp_payment_id;
const expected = crypto
.createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
.update(body).digest('hex');
```

```
if (expected !== rzp_signature)
return res.status(400).json({ message: 'Invalid signature' });

// Update payment and booking
await Payment.findOneAndUpdate(
{ rzp_order_id },
{ rzp_payment_id, rzp_signature, status: 'captured', paidAt: new Date() }
);
await Booking.findByIdAndUpdate(bookingId, { status: 'confirmed' });

// Send confirmation email (Phase 7)
await sendConfirmationEmail(req.user.email, bookingId);

res.json({ success: true, message: 'Booking confirmed' });
};
```

6.5 Razorpay Webhook (Backup Layer)

```
// routes/webhooks.js — must use express.raw() (not json parser) for Razorpay
router.post('/razorpay', express.raw({ type: 'application/json' }), async (req, res) =>
{
const signature = req.headers['x-razorpay-signature'];
const expected = crypto
.createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET)
.update(req.body).digest('hex');

if (expected !== signature) return res.status(400).send('Invalid');

const event = JSON.parse(req.body);
if (event.event === 'payment.captured') {
const rzp_order_id = event.payload.payment.entity.order_id;
// findAndUpdate if not already confirmed (idempotent)
await Payment.findOneAndUpdate({ rzp_order_id, status: { $ne: 'captured' } },
{ status: 'captured', paidAt: new Date() });
}
res.json({ received: true });
});
```

■ **The webhook route MUST use express.raw() middleware, NOT express.json(). The raw body bytes are required to verify the HMAC signature. Parsing it as JSON first will break signature verification.**

PHASE 7 · EMAIL SERVICE

Nodemailer + Gmail SMTP

IBEX Trekking — Standard Operating Procedure

7.1 Gmail SMTP Setup

1	**Enable 2-Step Verification on Gmail** Required before you can create an App Password.
2	**Generate App Password** Google Account → Security → 2-Step Verification → App Passwords → Select app: Mail, device: Other → name it 'IBEX Server' → Copy 16-char password.
3	**Add to .env** EMAIL_USER=ibex@gmail.com | EMAIL_PASS=<16-char app password>
4	**Alternative: Use SMTP service** For production consider SendGrid or AWS SES for higher sending limits and delivery tracking. Gmail is fine for up to ~500 emails/day.

7.2 Email Service (services/emailService.js)

```
const nodemailer = require('nodemailer');

const transporter = nodemailer.createTransport({
service: 'gmail',
auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASS },
});

exports.sendConfirmationEmail = async (toEmail, booking) => {
const mailOptions = {
from: `IBEX Trekking <${process.env.EMAIL_USER}>`,
to: toEmail,
subject: `Booking Confirmed — ${booking.trekId.title}`,
html: `
Your trek is booked!
Trek: ${booking.trekId.title}
Date: ${new Date(booking.trekDate).toDateString()}
Group Size: ${booking.groupSize}
Total Paid: ■${booking.totalAmount}
```

```
Booking ID: ${booking._id}

Our team will contact you 48 hours before the trek.
`,
};
await transporter.sendMail(mailOptions);
};

exports.sendContactEmail = async ({ name, email, message, trek }) => {
await transporter.sendMail({
from: `${name} <${email}>`,
to: process.env.EMAIL_USER,
subject: `New Enquiry — ${trek || 'General'}`,
html: `${message}From: ${name} (${email})`,
});
};
```

7.3 Email Templates to Build

• Booking confirmation (with trek details, guide contact, packing list link)

• Booking cancellation (with refund timeline)

• Trek reminder — sent 48 hours before trek date (via cron job / node-cron)

• Contact form auto-reply (immediate acknowledgement to user)

• Admin notification on new booking

• Password reset (if adding email/password auth later)

Next.js Pages & Components

8.1 Page Structure (App Router)

```
app/
■■■ (public)/
■ ■■■ page.jsx # Home
■ ■■■ about/page.jsx # About Us
■ ■■■ treks/
■ ■ ■■■ page.jsx # Trek listing + filters
■ ■ ■■■ [slug]/page.jsx # Trek detail + booking
■ ■■■ blog/
■ ■ ■■■ page.jsx # Blog listing
■ ■ ■■■ [slug]/page.jsx # Single blog post
■ ■■■ contact/page.jsx # Contact form
■■■ (auth)/
■ ■■■ login/page.jsx # Google sign-in button
■ ■■■ auth-callback/page.jsx # Handles OAuth redirect, stores session
■■■ (dashboard)/
■■■ dashboard/page.jsx # User bookings overview
■■■ admin/
■■■ treks/page.jsx # CRUD treks
■■■ bookings/page.jsx
■■■ blogs/page.jsx
```

8.2 Key Components

`Navbar.jsx`	Sticky, forest-dark bg. Shows Login or avatar + dropdown based on auth state.
`TrekCard.jsx`	Card with image, title, difficulty badge, duration/altitude/price in mono, 'View Trek' link.
`TrekFilter.jsx`	Region dropdown + difficulty pills + duration range slider. Updates URL params.
`BookingForm.jsx`	Date picker, group size input, price calculator. Calls POST /api/bookings then opens Razorpay modal.

`RazorpayButton.jsx`	Loads Razorpay JS script, opens checkout modal, calls POST /api/payments/verify on success.
`ImageUploader.jsx`	Drag-and-drop for admin. Sends multipart form to backend → Cloudinary.
`BlogEditor.jsx`	Rich text editor (react-quill or TipTap) for admin blog creation.
`ReviewForm.jsx`	Star rating widget + comment textarea. Protected route.
`ContactForm.jsx`	Name, email, phone, trek select, message. Calls POST /api/contact.
`ProtectedRoute.jsx`	HOC: checks auth context → redirects to /login if unauthenticated.

8.3 Razorpay Frontend Integration

```
// components/RazorpayButton.jsx
const loadRazorpay = () => new Promise(resolve => {
const script = document.createElement('script');
script.src = 'https://checkout.razorpay.com/v1/checkout.js';
script.onload = () => resolve(true);
document.body.appendChild(script);
});

const handlePayment = async () => {
await loadRazorpay();
// 1. Create booking + get rzp_order_id from our API
const { rzp_order_id, amount, bookingId } = await api.post('/bookings', bookingData);

// 2. Open Razorpay checkout
const options = {
key: process.env.NEXT_PUBLIC_RAZORPAY_KEY_ID,
amount: amount * 100,
currency: 'INR',
order_id: rzp_order_id,
name: 'IBEX Trekking',
handler: async (response) => {
// 3. Verify signature on our backend
await api.post('/payments/verify', { ...response, bookingId });
router.push('/dashboard?booked=true');
},
prefill: { name: user.name, email: user.email },
theme: { color: '#c8602a' }, // IBEX amber
```

```
};
new window.Razorpay(options).open();
};
```

Security Hardening + Prod Launch

9.1 Security Checklist

- ☐ helmet() middleware on all Express routes (sets secure HTTP headers)
- ☐ express-rate-limit on /api/auth/* (max 20 req/15min), /api/bookings (max 10/min)
- ☐ express-validator on all POST/PUT routes — validate and sanitize every input field
- ☐ CORS: whitelist only process.env.CLIENT_URL, not wildcard *
- ☐ Mongoose: never expose __v, use .select('-password -__v') on User queries
- ☐ Razorpay: always verify HMAC-SHA256 signature — never trust client-side success alone
- ☐ Cloudinary: restrict file types in upload preset (jpg, jpeg, png, webp only, max 5MB)
- ☐ JWT: use strong JWT_SECRET (min 256-bit random), set 7-day expiry, rotate on logout
- ☐ httpOnly cookie: set secure: true and sameSite: 'strict' in production
- ☐ MongoDB: never expose Atlas connection string in frontend or version control
- ☐ Add .env to .gitignore before first commit

9.2 Testing Strategy

1	**Unit tests (Jest)** Test each controller in isolation: createBooking, verifyPayment, createTrek. Mock MongoDB with jest.mock('../../models/Booking').
2	**Integration tests (Supertest)** Test full request/response for each route: POST /api/bookings should return 401 without JWT, 201 with valid JWT.
3	**Razorpay test mode** Use Razorpay test keys. Test card: 4111 1111 1111 1111, any future expiry, any CVV. Test UPI: success@razorpay.
4	**Email testing** Use Mailtrap (mailtrap.io) in dev to catch outgoing emails without delivering them. Update EMAIL_HOST in .env for dev.

5	**Manual QA checklist** Full user journey: sign in with Google → browse treks → filter → book → pay → check email → view dashboard → cancel booking.

9.3 Deployment

Backend	Railway.app or Render.com — connect GitHub repo, set all .env variables in dashboard, deploy automatically on push to main.
Frontend	Vercel — import GitHub repo, set NEXT_PUBLIC_API_URL and NEXT_PUBLIC_RAZORPAY_KEY_ID in Environment Variables.
MongoDB	MongoDB Atlas M0 (free) or M10 ($57/mo) for production. Enable Atlas Backups.
Domain	Buy ibex.in on GoDaddy/Namecheap. Point A record to Railway IP, CNAME 'www' to Vercel domain.
SSL	Vercel and Railway both auto-provision Let's Encrypt SSL. No manual setup needed.
Monitoring	Add Sentry for error tracking (sentry.io — free tier). Add UptimeRobot for uptime alerts.

PHASE A · APPENDIX

Environment Variables Reference

IBEX Trekking — Standard Operating Procedure

Backend — backend/.env

NODE_ENV	development | production
PORT	5000
MONGO_URI	mongodb+srv://:@cluster0.ibex.mongodb.net/ibex
JWT_SECRET	minimum 256-bit random string — never reuse
GOOGLE_CLIENT_ID	from Google Cloud Console → Credentials
GOOGLE_CLIENT_SECRET	from Google Cloud Console → Credentials
GOOGLE_CALLBACK_URL	http://localhost:5000/api/auth/google/callback
CLIENT_URL	http://localhost:3000
CLOUDINARY_CLOUD_NAME	your cloud name from Cloudinary dashboard
CLOUDINARY_API_KEY	from Cloudinary API Keys section
CLOUDINARY_API_SECRET	from Cloudinary API Keys section
RAZORPAY_KEY_ID	rzp_test_xxxxx (test) or rzp_live_xxxxx (prod)
RAZORPAY_KEY_SECRET	from Razorpay Dashboard → Settings → API Keys
RAZORPAY_WEBHOOK_SECRET	from Razorpay Dashboard → Settings → Webhooks
EMAIL_USER	your.gmail@gmail.com
EMAIL_PASS	16-character Gmail App Password

Frontend — frontend/.env.local

NEXT_PUBLIC_API_URL	http://localhost:5000/api
NEXT_PUBLIC_RAZORPAY_KEY_ID	rzp_test_xxxxx (same as backend KEY_ID)
NEXT_PUBLIC_CLOUDINARY_NAME	your cloud name (for direct browser uploads if needed)

■ **NEVER commit .env or .env.local to Git. Add both to .gitignore before your first commit. Use GitHub Secrets or Railway/Vercel dashboard for production variables.**

Project Timeline (Estimated)

Timeline	Phase	Deliverable
Week 1	Phase 1–2	Project setup, folder structure, MongoDB models, db connection
Week 2	Phase 3	Google OAuth 2.0, JWT middleware, auth routes, test login flow
Week 3	Phase 4–5	Trek/Blog/Review CRUD APIs + Cloudinary image upload
Week 4	Phase 6	Booking flow + Razorpay integration + webhook handler
Week 5	Phase 7–8	Nodemailer emails + Next.js frontend pages + components
Week 6	Phase 8	Admin dashboard, protected routes, responsive polish
Week 7	Phase 9	Testing, security hardening, bug fixes
Week 8	Launch	Deploy backend (Railway) + frontend (Vercel), domain setup, go live